Lecture 17: IPv4, IPv6, NAT, and ICMP

The network layer's protocols — what's inside every packet

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Identify and explain the purpose of each field in the **IPv4 datagram header**
- Explain how **ICMP** provides error reporting and diagnostics for IP
- Describe how `ping` and `traceroute` work at the protocol level
- Explain IP **fragmentation** and why it is problematic
- Describe how **NAT** works and why it complicates end-to-end communication
- Compare **IPv4** and **IPv6** headers and addressing
- Outline the **DHCP** process for dynamic address configuration

Key Acronyms

Acronym	Meaning
CIDR	Classless Inter-Domain Routing
DF	Don't Fragment
DHCP	Dynamic Host Configuration Protocol
DSCP	Differentiated Services Code Point
ECN	Explicit Congestion Notification
ICMP	Internet Control Message Protocol
IHL	Internet Header Length

Acronym	Meaning
MTU	Maximum Transmission Unit
NAT	Network Address Translation
NDP	Neighbor Discovery Protocol
PAT	Port Address Translation
PMTUD	Path MTU Discovery
SLAAC	Stateless Address Auto-Configuration
TTL	Time to Live

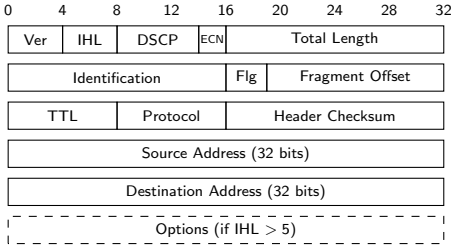
Callback: From Routing to the Datagram

In L13–L16 we built the routing picture:

- L13: Forwarding tables and longest-prefix match
- L14: IP addresses and CIDR prefixes
- L15: Dijkstra / link-state routing (OSPF)
- L16: Bellman-Ford / distance-vector (RIP) + BGP
- **L17: The datagram itself — IPv4, ICMP, NAT, IPv6**

We know *how* a router decides where to send a packet.
Today: what exactly *is* the packet? What fields does every router examine?

The IPv4 Header (20 bytes minimum)



What routers examine at each hop:

- Destination Address → forwarding decision
- **TTL** → decrement, drop if 0 (sends ICMP)
- Header Checksum → verify & recompute
- Total Length → packet boundary

What passes through unchanged:

- Source Address (exception: NAT)
- **Protocol** → demux key (1=ICMP, 6=TCP, 17=UDP)
- DSCP (unless QoS-aware router)
- Payload (data from upper layers)

Version & IHL are 4 bits each. Options rarely used today (IHL almost always 5 = 20-byte header).

Key Fields: TTL and Protocol

TTL — Time to Live (8 bits)

- Each router decrements by 1
- If TTL $\rightarrow$ 0: drop packet, send **ICMP Time Exceeded**
- Prevents packets from looping forever in routing loops
- We saw this in L13 with `traceroute`!

OS	Default TTL
Linux	64
Windows	128
Network equipment	255

Protocol (8 bits)

Tells the destination host which upper-layer module should receive the payload — the **demultiplexing key**.

Value	Protocol
1	ICMP
6	TCP
17	UDP
89	OSPF

Analogous to the **EtherType** field in Ethernet frames (L08).

DSCP: IP's QoS Mechanism

Recall from L13: the internet is fundamentally **best-effort**. DSCP provides a way to differentiate traffic *without* per-flow signaling.

DSCP	Per-Hop Behavior	Typical Use	Priority
000000	Best Effort (default)	Web browsing	Lowest
001010–011010	Assured Forwarding	Video streaming	Medium
101110	Expedited Forwarding	VoIP calls	Highest

How DSCP works

The sender (or edge router) **marks** packets with a DSCP value. Each router independently decides how to treat each class — prioritize in queues, drop lower-priority traffic first during congestion. **No per-flow state.** Scales to the core internet.

[Wikipedia: Differentiated services]

ICMP: The Internet's Error Reporter

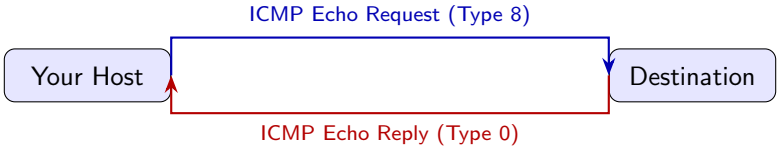
The **Internet Control Message Protocol (ICMP)** provides error reporting and diagnostics for IP. It is *part of the network layer* but carried inside IP datagrams (Protocol = 1).

Type	Code	Name	Purpose
0	0	Echo Reply	Response to ping
3	0	Dest Unreachable: Net	No route to network
3	1	Dest Unreachable: Host	Can reach net, not host
3	3	Dest Unreachable: Port	Host reached, port closed
3	4	Frag Needed, DF Set	Packet too big, can't fragment
8	0	Echo Request	Ping request
11	0	Time Exceeded: TTL	TTL decremented to 0

For error messages: body includes the **original IP header + first 8 bytes** of the offending packet (enough to identify the connection: src/dst ports).

[Wikipedia: Internet Control Message Protocol]

How ping Works — At the Protocol Level

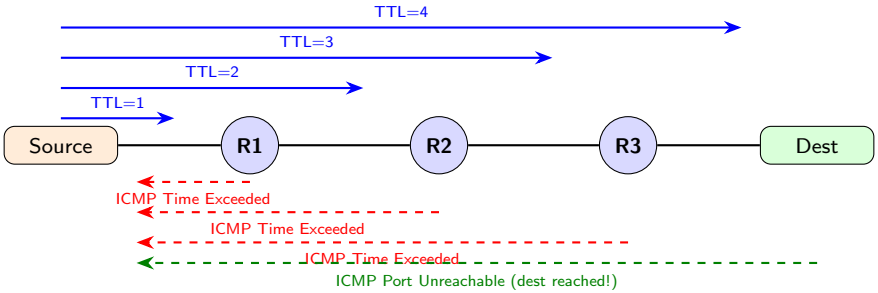


1. Host creates **ICMP Echo Request** (Type 8, Code 0) with identifier + sequence number
2. Encapsulated in IP datagram with Protocol = 1
3. Routed to destination through the forwarding machinery from L13–L16
4. Destination replies with **ICMP Echo Reply** (Type 0, Code 0)
5. Sender measures RTT

Key insight

ping does **not** use TCP or UDP. It operates directly at the network layer using ICMP.

How `traceroute` Works — At the Protocol Level



1. Send packet with **TTL = 1** → first router sends back **ICMP Time Exceeded**
2. Increment TTL each round → discover each router along the path
3. Destination responds with **ICMP Port Unreachable** (Unix) or **Echo Reply** (Windows)

* * * in output = that router does not send ICMP replies (rate-limited or blocked).

IP Fragmentation

Problem: links have different **Maximum Transmission Units (MTUs)**.

Link Type	MTU
Ethernet	1,500 B
PPPoE (DSL)	1,492 B
WiFi (802.11)	2,304 B
Jumbo frames	9,000 B

When a datagram is too large for the outgoing link, the router can:

1. **Fragment** it into smaller pieces
2. **Drop** it and send ICMP error (if DF flag is set)

Example: 4,000-byte datagram entering an Ethernet link (MTU = 1,500):

Fragment	Total Length	MF Flag	Offset	Payload
1	1,500	1	0	Bytes 0–1,479
2	1,500	1	185	Bytes 1,480–2,959
3	1,040	0	370	Bytes 2,960–3,979

All fragments share the same **Identification** value. Offset is in **8-byte units**. Reassembly happens only at the **destination**.

Why Fragmentation Is Problematic

Problems:

1. **Losing one fragment = losing the entire datagram** — the other fragments wasted bandwidth
2. **Reassembly is complex** — buffering, ordering, timeout handling at destination
3. **Security vulnerabilities** — overlapping fragments exploited in attacks (Ping of Death, Teardrop)
4. **Header overhead** — each fragment carries its own 20-byte IP header

Modern solution: Path MTU Discovery

1. Sender sets **DF (Don't Fragment)** flag
2. If packet too big, router sends **ICMP Fragmentation Needed** (Type 3, Code 4) including the link MTU
3. Sender reduces packet size and retries
4. Repeat until the **path MTU** is found

Caution

Firewalls that block all ICMP break PMTUD — creates "black holes" where large packets silently disappear.

[Wikipedia: Path MTU Discovery]

Demo: Fragmentation and ICMP

Demo: triggering fragmentation and observing ICMP

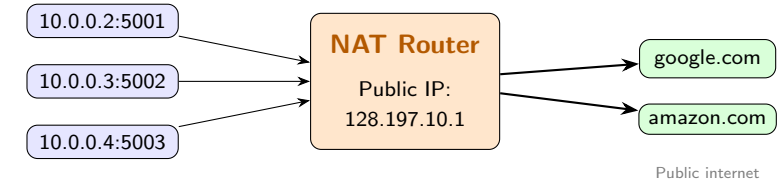
```
# Send a large ping that exceeds Ethernet MTU
$ ping -s 3000 -c 1 google.com
# => fragmented into multiple IP datagrams

# Try with Don't Fragment set
$ ping -D -s 2000 -c 1 google.com
# => "Message too long" (ICMP Frag Needed)

# In Wireshark: filter on 'icmp'
# Observe:
# Type 8 (Echo Request) going out
# Type 0 (Echo Reply) coming back
# Protocol field = 1 in IP header
```

L13 callback: We used `ping` and `traceroute` as black boxes. Now we know the ICMP protocol mechanics underneath.

NAT: The Hack That Saved the Internet

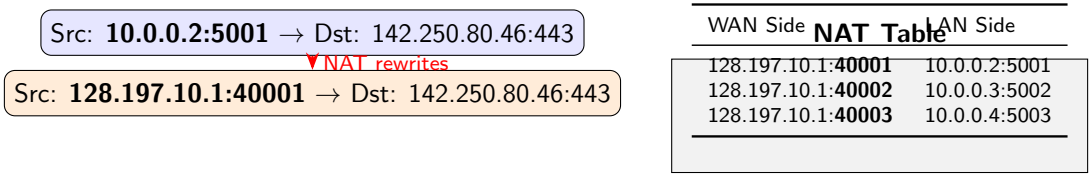
Problem: IPv4 has $2^{32} \approx 4.3$ billion addresses, but far more devices.

Solution: **Network Address Translation (NAT)** — an entire private network shares **one public IP address**.



NAT replaces the **source IP and port** on outgoing packets, records the mapping, and **restores them** on replies.

NAT Translation Table



This is **Port Address Translation (PAT)** — the most common form.

- A single public IP supports up to ∼65,000 simultaneous connections
- NAT modifies **both** the IP header (addresses) **and** transport header (ports)
- Must recompute IP **and** TCP/UDP checksums

[Wikipedia: Network address translation]

NAT and the End-to-End Principle

NAT fundamentally **violates the end-to-end principle**:

What breaks:

- **Incoming connections fail** — external hosts cannot reach private addresses
- **Peer-to-peer is complicated** — two hosts behind NAT cannot directly communicate
- **Embedded IPs break** — protocols like FTP, SIP embed addresses in payload

Workarounds:

- Port forwarding (manual)
- UPnP / NAT-PMP (automatic)
- STUN / TURN / ICE (WebRTC, VoIP)
- Hole punching (gaming, P2P)

NAT: love it or hate it

- ✓ Saved the internet from IPv4 exhaustion
- ✓ De facto security (not reachable from outside)
- × Breaks clean end-to-end design
- × **Made IPv6 adoption stall** — NAT made IPv4 "good enough"

IPv6: The Original Vision (1990s)

Designed to solve the IPv4 address exhaustion crisis:

- **Vast address space**: 128 bits $\rightarrow 2^{128} \approx 3.4 \times 10^{38}$ addresses
- **Global addressability**: every device gets a real, routable address — no NAT
- **Simplified header**: fixed 40 bytes, no checksum, faster router processing
- **No fragmentation in transit**: only endpoints fragment
- **Auto-configuration (SLAAC)**: hosts configure themselves without DHCP
- **Mandatory IPsec**: built-in encryption for every connection

The plan

IPv4 would be deprecated. The entire internet would transition to IPv6.
That was **1998** — nearly 30 years ago.

[Wikipedia: IPv6]

IPv6: The Reality Check

Why adoption stalled:

- **NAT extended IPv4's life** by ∼20 years
- Enterprise: IPv4 + NAT "works fine" — upgrade cost > perceived benefit
- Dual-stack infrastructure is expensive
- Applications need testing on IPv6
- "If it ain't broke..."

Who actually deployed IPv6:

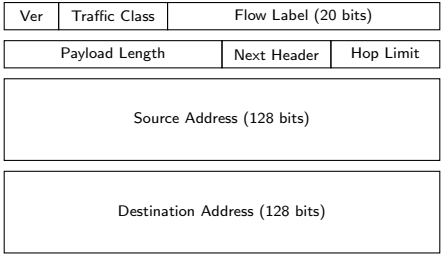
- **Mobile carriers** (ran out of IPv4 first):
  - T-Mobile US: >90% IPv6
  - Reliance Jio: IPv6-first
  - Verizon, AT&T: significant
- Your phone likely has a **real global IPv6 address** — no NAT

Current state (2025)

∼45% of Google traffic arrives over IPv6, but **distribution is uneven**: India, France, Germany > 60%; many countries < 10%. IPv6 is largely a **mobile-internet** story, not the universal enterprise transition that was envisioned.

[Wikipedia: IPv6 deployment]

IPv6 Header: Simplified



Fixed 40 bytes — no variable-length options in the base header.

What changed from IPv4:

- **Removed**: Header checksum (redundant), fragmentation fields, IHL, options
- **Renamed**: TTL → **Hop Limit**, Protocol → **Next Header**
- **Added**: Flow Label (20 bits) for identifying packet flows
- Optional features moved to **extension headers** (linked list via Next Header)

Version is always 6. Next Header serves the same role as IPv4's Protocol field (1=ICMP, 6=TCP, 17=UDP) but also chains extension headers.

IPv4 vs. IPv6: Side by Side

Feature	**IPv4**	**IPv6**
Address size	32 bits (4.3B addresses)	128 bits (3.4×10^{38})
Header size	20–60 bytes (variable)	40 bytes (fixed)
Header checksum	Yes (recomputed every hop)	**Removed**
Fragmentation	In base header; routers fragment	**Removed**; only source fragments
Options	In base header (variable IHL)	Extension headers
TTL / Hop Limit	Time to Live	Hop Limit (renamed)
Broadcast	Yes (255.255.255.255)	**No broadcast** — multicast only
Auto-config	DHCP required	SLAAC (stateless)

Why remove the header checksum?

Recomputing it at every hop wastes CPU. Link-layer CRCs catch bit errors per link. Transport-layer checksums (TCP/UDP) catch end-to-end corruption. The IP header checksum was **redundant work** at every router.

IPv6 Address Notation

Full form: eight groups of four hex digits, separated by colons:

`2001:0db8:85a3:0000:0000:8a2e:0370:7334`

Compression rules (applied in order):

1. **Drop leading zeros** in each group: `0db8` → `db8`, `0370` → `370`, `0000` → `0`
   Result: `2001:db8:85a3:0:0:8a2e:370:7334`
2. **Replace one longest run of consecutive all-zero groups with `::`**
   Result: `2001:db8:85a3::8a2e:370:7334`

The `::` rule

`::` can appear **only once** per address — otherwise the expansion would be ambiguous (you couldn't tell how many zero groups each `::` replaced). When expanding, count the groups present and fill in zeros to make 8 total.

Examples: `::1` = loopback (7 groups of zeros + 1). `fe80::1` = `fe80:0:0:0:0:0:0:1`. `2001:db8::1:0:0:1` = `2001:0db8:0000:0000:0001:0000:0000:0001` (6 zeros filled in? No — 4 groups present besides `::`, so `::` expands to 4 zero groups).

IPv6 Address Types and Allocation

Type	Prefix	Scope	Purpose
Global Unicast	`2000::/3`	Internet	Globally routable (= public IPv4)
Link-Local	`fe80::/10`	Single link	Auto-configured; neighbor discovery
Unique Local	`fc00::/7`	Organization	Like RFC 1918 private addresses
Multicast	`ff00::/8`	Varies	**Replaces broadcast**
Loopback	`::1/128`	Host	Equivalent to 127.0.0.1

Standard allocation hierarchy:

/32 from RIR to ISP (or **/29–/32** for large ISPs)

/48 from ISP to customer site (standard enterprise block)

/64 per subnet (required for SLAAC to work)

- A **/48** gives an organization $2^{16} = 65{,}536$ subnets, each a /64
- A **/64** gives each subnet $2^{64} \approx 1.8 \times 10^{19}$ host addresses
- The /64 boundary is *mandatory* — SLAAC and Neighbor Discovery assume it

IPv6 Transition Mechanisms

The internet cannot switch in a day — there is no "flag day."

Dual Stack

Hosts run **both** IPv4 and IPv6. DNS returns both A and AAAA records. "Happy Eyeballs" algorithm (RFC 8305) prefers IPv6 when available.

Most common approach today.

Tunneling

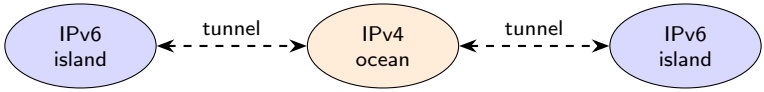
IPv6 packets **encapsulated inside IPv4** to cross IPv4-only segments. IPv6 islands communicate across an IPv4 ocean.

6in4, 6to4, Teredo.

NAT64 / DNS64

Translates between IPv6-only clients and IPv4-only servers. Maps IPv4 addresses into a special IPv6 prefix (`64:ff9b::/96`).

Used by mobile carriers running IPv6-only.



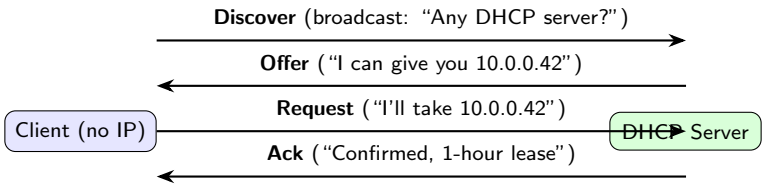
Demo: IPv6 on Your Machine

Demo: seeing IPv4 and IPv6 addresses side by side

```
# See both v4 and v6 addresses
$ ifconfig en0
  inet 192.168.1.42 <- IPv4 (behind NAT)
  inet6 fe80::1a2b:3c4d <- link-local (always)
  inet6 2601:1234:abcd::42 <- global (if ISP supports v6)

# Ping over IPv6
$ ping6 google.com
PING6(56=40+8+8 bytes) ... -> 2607:f8b0:4006:80e::200e
round-trip min/avg/max = 12.3/13.1/14.2 ms

# Compare: your phone likely has a real global
# IPv6 address from your mobile carrier -- no NAT!
```

DHCP: How Hosts Get Their Addresses

Problem: a new host has no IP, no subnet mask, no gateway, no DNS server.



- Uses **UDP** (port 67 server, port 68 client) — can't use TCP with no IP address
- Client sends from **0.0.0.0** to **255.255.255.255** (broadcast)
- **Leases** expire — client must renew or address returns to pool
- **Relay agents**: routers forward DHCP broadcasts across subnets

IPv6: SLAAC handles addresses; DHCPv6 provides DNS and other options.

[Wikipedia: DHCP]

L17 Summary: The Network Layer's Protocols

Topic	Key Takeaway
IPv4 header	20-byte base with TTL, Protocol, addresses; DSCP for QoS
ICMP	Error reporting (Dest Unreachable, Time Exceeded) and diagnostics (ping, traceroute) — the protocol behind L13's demos
Fragmentation	Problematic — PMTUD avoids it; IPv6 bans router fragmentation
NAT	Saved IPv4 but broke end-to-end; uses port translation
IPv6	128-bit addresses, simplified header, no broadcast; mobile-driven adoption
DHCP	Discover/Offer/Request/Ack — how hosts bootstrap onto a network

The Network Layer Is Complete

L13: Forwarding and routing (data plane / control plane)

L14: IP addressing, CIDR, subnetting

L15: Link-state routing / Dijkstra (OSPF)

L16: Distance-vector / Bellman-Ford (RIP) + BGP

L17: IPv4, ICMP, NAT, IPv6, DHCP (today)

Coming up:

1. **L18 (Thu Apr 02)**: Wireshark lab — hands-on packet analysis (L13–L17)
2. Then: sliding window, TCP, and the **transport layer**

Bring your laptop with **Wireshark** installed for L18. Pre-recorded pcaps will be provided for protocols we can't capture live.